VR 引导的人形机器人共享遥操作系统

基于 Meta Quest 3 与 Hand Tracking Streamer 的技术方案、安装运行与集成路线

项目阶段	论文初期方案论证与原型搭建
核心设备	Meta Quest 3
核心软件	Hand Tracking Streamer / Python / ROS 2
目标平台	仿真人形机器人及后续实体机器人
文档用途	项目讨论、原型实施与 Proposal 扩展

Working Title: VR-Guided Target Pose Control with Autonomous Whole-body Stabilization for Humanoid Robots

版本：项目框架与工程实现文档

摘要

本项目面向人形机器人的沉浸式远程操作，提出一种“直接控制 + 命令控制 + 自主全身控制”的分层共享遥操作框架。操作员佩戴 Meta Quest 3，在坐姿条件下连续控制机器人头部和双手目标位姿，并通过语音与手势提供行走、转向、抓取、下蹲、模式切换等离散命令。机器人不直接复现人体全部关节运动，而是结合视觉感知、可达性分析、碰撞检测、步态规划、全身逆运动学以及 WBC/MPC，自主生成满足稳定性和动力学约束的全身运动。

在原型阶段，Hand Tracking Streamer（HTS）作为人体运动输入层，负责从 Quest 3 获取左右手腕 6DoF 位姿、每只手 21 个关键点以及头部位姿，并通过 TCP 或 UDP 实时传输至电脑。文档系统整理 HTS 的数据结构、安装与运行方法、坐标系转换、ROS 2 接口设计、仿真集成路线、实验指标和阶段性实施计划，为后续形成 8–12 页正式研究 Proposal 提供工程基础。

1. 项目背景与研究动机

传统人形机器人遥操作通常采用全身动作重定向或低层关节控制。前者要求操作员持续提供全身动作，认知和体力负担较高；后者对机器人模型、关节约束和控制经验要求过高。在双足机器人场景中，若直接将人体腿部运动映射到机器人，容易因尺度差异、动力学差异、通信延迟和环境约束导致失稳。

本项目的核心判断是：人应负责表达任务意图和关键目标位姿，机器人应负责完成稳定、可行和安全的运动执行。因此，头部和双手采用连续直接控制，腿部、骨盆、躯干、接触切换、行走和平衡由机器人自主完成。该范式将“操作者控制每个关节”转变为“操作者定义机器人应达到什么状态，机器人自主决定如何达到”。

2. 核心概念与研究目标

2.1 核心控制范式

```
Direct Control + Command Control + Autonomous Whole-body Control

Direct Control:
    Head target pose
    Left-hand target pose
    Right-hand target pose

Command Control:
    Walk / Turn / Stand / Squat / Grasp / Release / Navigate
```

```
Autonomous Control:
    Locomotion / Balance / Torso-pelvis coordination
    Collision avoidance / Contact planning / Whole-body stabilization
```

2.2 总体研究目标

- 建立基于 Meta Quest 3 的沉浸式人体意图获取接口。
- 设计 Target Pose Manager，将连续 VR 位姿与离散语音、手势命令统一为机器人任务目标。
- 利用视觉感知完善目标位姿、环境约束、碰撞信息、可达性和可通行性。
- 实现从用户目标位姿到稳定全身运动的自主规划与控制链路。
- 通过实验验证共享控制对任务成功率、稳定性和操作员负担的改善。

3. 系统总体架构

```
Human Operator
    ↓
Meta Quest 3: Head & Hand Tracking + Gesture + Speech
    ↓
Human Intention Layer
    ↓
Target Pose Manager
    ↓
Vision Perception + Robot State Estimation
    ↓
Autonomous Whole-body Planner
    ↓
Whole-body Controller (IK / WBC / MPC)
    ↓
Humanoid Robot
    ↓
First-person Video + Robot State Feedback
    └──────────────────────────────→ Operator
```

3.1 功能模块

模块	主要输入	主要输出	关键功能
人体交互层	Quest 头手位姿、语音、手势	原始人体意图数据	追踪、识别、时间戳同步
Target Pose Manager	连续位姿、离散命令	统一目标位姿与任务状态	融合、模式管理、尺度映射、安全限幅
环境感知层	机器人相机、深度、状态	目标、障碍、可达性、地形信息	检测、场景理解、可通行性和 affordance 分析
全身运动规划层	目标位姿、机器人状态、环境约束	足步、接触序列、全身轨迹	IK、足步规划、碰撞规避、平衡规划
全身控制层	参考轨迹、状态估计	关节力矩/位置命令	WBC、MPC、动态稳定执行
沉浸式反馈层	机器人视频和状态	VR 第一视角与增强信息	状态可视化、告警、任务反

模块	主要输入	主要输出	关键功能
			馈

4. Hand Tracking Streamer 在本项目中的定位

Hand Tracking Streamer（HTS）是运行在 Meta Quest 设备上的实时手部追踪与网络传输应用。它已经完成 Quest 端 OpenXR/Meta Interaction SDK 数据读取、手部骨架获取和网络发送，因此可直接作为本项目的人体运动输入层，显著降低原型阶段的开发成本。

```
Meta Quest 3
    ↓  Hand and head tracking
Hand Tracking Streamer App
    ↓  TCP / UDP
Computer Receiver
    ├── Python Visualizer
    ├── Python SDK
    ├── ROS 2 Node
    └── Robot Controller
```

4.1 输出数据

- 左右手腕 6DoF 位姿：位置 x, y, z 与四元数 qx, qy, qz, qw。
- 每只手 21 个关键点，每个关键点包含三维坐标，共 63 个浮点数。
- 头部位姿，可用于机器人头部目标姿态和视角控制。
- 支持单手或双手、TCP 或 UDP、Quest 内骨架可视化和网络调试。

4.2 数据到机器人控制目标的映射

```
Quest right wrist pose
    ↓ Coordinate calibration
Robot right end-effector target pose
    ↓ Workspace limiting and smoothing
Whole-body IK / WBC
    ↓
Robot arm joint commands

Finger landmarks
    ↓
Pinch / open hand / pointing / grasp detection
    ↓
Gripper command or high-level task command
```

4.3 HTS 不负责的部分

- 机器人 URDF、关节约束和动力学模型。
- 从手腕位姿到机器人关节角的逆运动学。
- 碰撞检测、工作空间限制、运动缩放和安全滤波。
- 双足机器人的足步规划、平衡控制和全身控制。
- 机器人通信驱动和闭环状态反馈。

5. 推荐的首次运行方案

第一次测试应以验证数据链路为目标，不建议立即修改 Unity 工程或接入 ROS 2。推荐采用 Quest Store/预编译 APK + USB 有线 TCP + Python visualizer。该方案稳定性高，便于排除 Wi-Fi、防火墙和 UDP 抖动等因素。

```
Quest Store / APK installation
    ↓
Quest 3 connected to computer through USB-C
    ↓
ADB reverse TCP port mapping
    ↓
Python visualizer on computer
    ↓
Verify left/right wrist, fingers and head tracking
```

6. 电脑端安装与环境准备

6.1 下载仓库

```
git clone https://github.com/wengmister/hand-tracking-streamer.git
cd hand-tracking-streamer
```

6.2 创建 Python 虚拟环境

```
# Ubuntu / macOS
python3 -m venv .venv
source .venv/bin/activate

# Windows PowerShell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

6.3 安装依赖

```
pip install --upgrade pip
pip install -e .
pip install hand-tracking-sdk numpy matplotlib
```

如出现 GUI 或 OpenGL 相关模块缺失，可根据报错补充安装 pyqt5、pyqtgraph 等依赖。Python 3.10 或 3.11 通常更适合作为原型环境。

7. Quest 3 与 ADB 配置

1. 在 Meta 手机应用中关联 Quest 3，并创建或加入 Meta Developer Organization。
2. 开启 Quest Developer Mode，重启设备。

3. 安装 Android Platform Tools / ADB。
4. 使用支持数据传输的 USB-C 线连接 Quest 3 与电脑。
5. 在头显内批准“Allow USB debugging”，建议勾选始终允许。

```
adb version
adb devices
```

正常状态应显示设备序列号并标记为 device。若显示 unauthorized，需要重新在 Quest 3 内批准 USB 调试。

8. USB 有线 TCP 运行流程

8.1 设置反向端口映射

```
adb reverse tcp:8000 tcp:8000
adb reverse --list
```

该映射使 Quest 应用访问 localhost:8000 时，经 USB 连接到电脑的 8000 端口。

8.2 启动电脑端接收程序

```
# Ubuntu / macOS
python ./scripts/visualizer.py \
  --protocol tcp \
  --host 0.0.0.0 \
  --port 8000 \
  --show-fingers

# Windows PowerShell
python .\scripts\visualizer.py --protocol tcp --host 0.0.0.0 --port 8000 --show-fingers
```

8.3 Quest 端配置

配置项	推荐值
Protocol	TCP
IP Address / Host	localhost
Port	8000
Hand Side	Both
Visualization	On

8.4 正确启动顺序

6. 连接 Quest 3 与电脑。
7. 运行 adb devices。
8. 运行 adb reverse tcp:8000 tcp:8000。

9. 启动电脑端 visualizer。
10. 打开 Quest 中的 HTS。
11. 设置 TCP、localhost、8000。
12. 点击 Start Streaming。

9. 无线 TCP 与 UDP

9.1 无线 TCP

Quest 3 与电脑必须连接同一局域网。Quest 中填写电脑的本地 IPv4 地址，电脑端监听 0.0.0.0。

```
# 查询电脑 IP
hostname -I          # Ubuntu
ipconfig             # Windows
ipconfig getifaddr en0   # macOS

python ./scripts/visualizer.py --protocol tcp --host 0.0.0.0 --port 9000 --show-fingers
```

9.2 无线 UDP

```
python ./scripts/visualizer.py --protocol udp --host 0.0.0.0 --port 9000 --show-fingers
```

UDP 可采用电脑局域网 IP，也可使用默认广播地址 255.255.255.255:9000。其潜在延迟较低，但 Wi-Fi 省电、DTIM 和拥塞可能造成批量发送与延迟尖峰。对机器人实时控制，优先选择 USB TCP 或稳定的无线 TCP。

10. 防火墙与故障排查

现象	可能原因	检查与处理
adb devices 为空	开发者模式未启用、线缆仅充电、驱动问题	确认 Developer Mode；更换数据线；重新连接 USB；Windows 检查 ADB 驱动
设备显示 unauthorized	未批准 USB 调试	在 Quest 中点击 Allow；必要时撤销 USB debugging 授权后重连
TCP 连接失败	接收端未先启动、端口映射缺失	先启动 visualizer；检查 adb reverse --list；核对端口
无线模式无法连接	IP 地址错误、不同局域网、防火墙阻止	确认同一 Wi-Fi；核对 IPv4；允许 TCP/UDP 入站端口
数据抖动或延迟尖峰	Wi-Fi 拥塞、UDP 批量传输	改用 USB TCP；关闭省电；使用 5GHz/6GHz 网络

现象	可能原因	检查与处理
手部骨架不显示	Quest 手部追踪未启用或控制器仍在使用	开启 Hand Tracking；放下控制器；保证光照充足

11. 坐标系转换与标定

HTS/Unity 通常采用左手坐标系，而 ROS 和多数机器人系统采用右手坐标系。位置坐标不能直接复制，旋转四元数也不能仅通过交换元素完成转换。必须建立明确的坐标变换链。

```
Quest world frame
    ↓  T_base_from_vr
Robot base_link frame
    ↓  target pose mapping
Robot end-effector frame
    ↓  T_tool_offset
Robot tool / hand frame
```

11.1 位置映射示例

假设 Quest 坐标定义为 X 向右、Y 向上、Z 向前，机器人 base_link 定义为 X 向前、Y 向左、Z 向上，一种常见的初始映射为：

```
robot_x = quest_z
robot_y = -quest_x
robot_z = quest_y
```

该映射仅作为初始假设，最终应通过实际移动测试确定轴方向、符号和尺度。

11.2 旋转与工具偏置

```
R_robot = R_base_from_vr · R_vr · R_tool_offset
```

- R_base_from_vr：VR 世界坐标到机器人基坐标的旋转。
- R_vr：HTS 接收到的手腕旋转。
- R_tool_offset：人体手腕坐标系到机器人末端执行器坐标系的固定偏置。

11.3 推荐标定流程

13. 定义操作员中立姿态：双手自然前伸，头部朝前。
14. 记录左右手腕和头部的初始位姿。
15. 定义机器人对应的中立末端位姿。
16. 计算平移偏置、旋转偏置和尺度系数。
17. 分别测试沿 Quest X/Y/Z 方向移动，验证机器人映射方向。

18. 加入工作空间裁剪、速度限制、低通滤波和异常跳变抑制。

12. ROS 2 集成设计

12.1 推荐节点结构

```
hts_receiver_node
    ↓ raw tracking messages
vr_calibration_node
    ↓ calibrated poses
intent_manager_node
    ↓ unified target poses and commands
whole_body_planner_node
    ↓ feasible whole-body references
robot_controller_node
    ↓ joint commands
humanoid robot / simulator
```

12.2 推荐 Topic

Topic	消息类型	用途
/vr/left_wrist_pose	geometry_msgs/msg/PoseStamped	左手腕目标位姿
/vr/right_wrist_pose	geometry_msgs/msg/PoseStamped	右手腕目标位姿
/vr/head_pose	geometry_msgs/msg/PoseStamped	头部目标姿态
/vr/left_hand_landmarks	自定义消息或 MarkerArray	左手 21 关键点
/vr/right_hand_landmarks	自定义消息或 MarkerArray	右手 21 关键点
/vr/gesture_command	std_msgs/msg/String 或自定义枚举	手势命令
/vr/speech_command	std_msgs/msg/String 或自定义枚举	语音命令
/teleop/target_state	自定义 TargetState	统一目标与模式

12.3 Target Pose Manager 设计

- 连续目标：头部、左手和右手位姿。
- 离散目标：行走、转向、站立、下蹲、抓取、释放、导航、确认和模式切换。
- 状态机：Idle、Manipulation、Locomotion、Navigation、Emergency Stop。
- 安全机制：工作空间限制、最大速度、最大角速度、碰撞预检查和命令超时。
- 优先级管理：急停 > 安全约束 > 平衡约束 > 命令目标 > 连续手部目标。

13. 从原型到机器人系统的实施路线

阶段	技术链路	完成标准
Demo 1：数据链路	Quest 3 → HTS → USB TCP → Python Visualizer	左右手、手指和头部数据连续更新，无频繁断连。
Demo 2：原始数据读取	Python SDK 输出位姿和关键点	可记录时间戳、频率、延迟和丢包情况。
Demo 3：ROS 2 发布	HTS Receiver → ROS 2 Topics → RViz	坐标系、消息类型和可视化正确。
Demo 4：单臂仿真	VR 目标 → IK → MuJoCo/Isaac Sim/Gazebo	机械臂末端能稳定跟随，满足关节限制。
Demo 5：双臂与头部	双手+头部 → Whole-body IK	双臂协同且无自碰撞。
Demo 6：共享控制	目标位姿 → 碰撞规避/可达性/平衡约束 → WBC	机器人自主调整躯干和足步完成任务。
Demo 7：闭环反馈	机器人第一视角视频和状态返回 Quest	形成完整沉浸式闭环遥操作。

14. 初始研究问题

19. 如何将 VR 连续位姿、语音命令和手势命令融合为统一、稳定且可解释的目标表示?
20. 如何从用户定义的头部和双手目标位姿，自主生成满足动力学、接触和稳定性约束的全身运动?
21. 如何利用视觉感知改善目标修正、可达性判断、碰撞规避、足步规划和动态稳定性?
22. 如何在保持操作直观性与末端精度的同时，降低操作员的认知和体力负担?
23. 在通信延迟、追踪噪声和短时丢失情况下，如何保持系统安全、连续和稳定?

15. 实验设计与评价指标

15.1 对比方法

- 纯 VR 直接遥操作：尽可能直接映射用户运动，机器人自主能力较弱。
- 全身动作重定向：人体全身动作映射到机器人。
- 本文提出的分层共享遥操作：头手直接控制 + 高层命令 + 自主全身稳定。

15.2 客观指标

- 任务完成率、任务完成时间、失败类型。
- 手部末端位置与姿态误差。
- 机器人稳定裕度、质心/零力矩点偏差、跌倒次数。
- 碰撞次数、关节限位触发次数、不可达目标比例。

- 端到端延迟、更新频率、抖动、丢包率。
- 自主修正比例、足步调整次数和操作员命令数量。

15.3 主观指标

- NASA-TLX 操作负荷。
- System Usability Scale 或自定义可用性问卷。
- 沉浸感、控制直观性、信任度和晕动不适。

16. 主要技术风险与应对策略

风险	影响	应对策略
Quest 追踪噪声或短时丢失	目标跳变、机械臂抖动	滤波、速度限制、置信度门控、保持最后有效目标、超时回中立位
坐标系和工具方向错误	末端运动方向不符合预期	离线标定、轴向单独测试、TF 树可视化、固定工具偏置
目标位姿不可达	IK 失败或关节超限	工作空间投影、可达性估计、软约束 IK、躯干与足步自主调整
通信延迟和抖动	控制滞后、闭环不稳定	时间戳、缓冲、预测、USB TCP 优先、本地安全控制
双足稳定与手部目标冲突	机器人失稳	任务优先级 WBC、平衡约束高优先级、自动降低手部跟踪增益
系统集成复杂度高	难以按期形成研究成果	采用分阶段 Demo，先单臂仿真，再逐步加入双臂、视觉和双足

17. 建议里程碑

阶段	核心任务	交付物
M1：输入链路	运行 HTS，测量频率、延迟和稳定性	可重复运行的 Quest-PC 数据链路
M2：ROS 2 接口	接收、标定、发布头手数据	ROS 2 包、TF 和 RViz 演示
M3：单/双臂仿真	IK、限幅、滤波、碰撞约束	VR 控制仿真机械臂演示
M4：Target Pose Manager	融合姿态、手势和语音，设计状态机	统一目标消息和控制模式
M5：共享全身控制	加入躯干、足步、平衡和 WBC	稳定完成伸手/抓取等任务
M6：视觉与评估	环境感知、对比实验、用户研究	实验数据、结果分析和论文材料

18. 预期贡献

- 一种面向人形机器人的分层 VR 共享遥操作系统架构。
- 一个融合连续头手位姿与离散语音/手势命令的 Target Pose Manager。
- 从用户目标位姿到视觉引导、自主稳定全身运动的控制链路。
- 一套基于 Meta Quest 3、HTS、ROS 2 和人形机器人仿真的可复现实验平台。
- 关于共享控制、纯遥操作和全身重定向在性能、稳定性和操作负荷方面的定量比较。

19. 结论

该方案将 Meta Quest 3 作为直观的人体意图输入设备，将 Hand Tracking Streamer 作为快速可靠的数据采集与传输层，并把论文研究重点放在 Target Pose Manager、视觉约束、全身运动生成与共享自治上。工程上应从“能稳定接收头手数据”开始，依次完成 ROS 2 接口、坐标标定、仿真机械臂跟随、双臂与头部控制、全身稳定以及闭环 VR 反馈。研究上，该项目的关键价值不在于简单复制人体动作，而在于建立一种由人定义目标、由机器人自主完成稳定执行的遥操作范式。

附录 A：Ubuntu 首次运行命令清单

```
sudo apt update
sudo apt install git adb python3-venv

git clone https://github.com/wengmister/hand-tracking-streamer.git
cd hand-tracking-streamer

python3 -m venv .venv
source .venv/bin/activate

pip install --upgrade pip
pip install -e .
pip install hand-tracking-sdk numpy matplotlib

adb devices
adb reverse tcp:8000 tcp:8000
adb reverse --list

python ./scripts/visualizer.py \
  --protocol tcp \
  --host 0.0.0.0 \
  --port 8000 \
  --show-fingers
```

附录 B：Quest 端首次运行配置

```
Protocol: TCP
Host: localhost
Port: 8000
Hand Side: Both
Visualization: On
Action: Start Streaming
```

附录 C：参考资源

- Hand Tracking Streamer GitHub Repository: https://github.com/wengmister/hand-tracking-streamer
- Hand Tracking Streamer CONNECTIONS.md: repository documentation for TCP/UDP, data format and coordinate notes.
- Meta Quest Developer Mode and Android Platform Tools documentation.
- ROS 2 geometry_msgs, visualization_msgs and TF2 documentation.
- MoveIt 2 / Pinocchio / MuJoCo / Isaac Sim / Gazebo documentation for subsequent integration.